

TALLER PROGCOMP: TRACK STRINGS

KNUTH-MORRIS-PRATT ALGORITHM

Gabriel Carmona Tabja

Universidad Técnica Federico Santa María,
Università di Pisa

April 13, 2025

Part I

FUNCIÓN PREFIJO

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(0) = 0$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aa**b**aaab

$$\pi(1) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(1) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(1) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aa**b**aaab

$$\pi(1) = \max\{0, 1\} = 1$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(2) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(2) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(2) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(2) = \max\{0\} = 0$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaab

$$\pi(3) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(3) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(3) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(3) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(3) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(3) = \max\{0, 1\} = 1$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaab

$$\pi(4) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(4) = \max\{0, 1, 2\} = 2$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(5) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aa**b**aaab

$$\pi(5) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aa**b**aaab

$$\pi(5) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(5) = \max\{0, 1\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(5) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaab

$$\pi(5) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(5) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(5) = \max\{0, 1, 2\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(5) = \max\{0, 1, 2\} = 2$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0, 3\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0, 3\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0, 3\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0, 3\}$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$$\pi(6) = \max\{0, 3\} = 3$$

FUNCIÓN PREFIJO

Definición

Dado un string s de largo n , se define la función siguiente:

$$\pi(i) = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Ejemplo

aabaaab

$\pi(i)$	0	1	0	1	2	2	3
i	0	1	2	3	4	5	6

FUNCIÓN PREFIJO - TRIVIAL ALGORITHM

```
1  vector<int> prefix_function(string s) {  
2      int n = (int)s.length();  
3      vector<int> pi(n);  
4      for (int i = 0; i < n; i++)  
5          for (int k = 0; k <= i; k++)  
6              if (s.substr(0, k) == s.substr(i-k+1, k))  
7                  pi[i] = k;  
8      return pi;  
9  }
```

FUNCIÓN PREFIJO - TRIVIAL ALGORITHM

```
1 vector<int> prefix_function(string s) {  
2     int n = (int)s.length();  
3     vector<int> pi(n);  
4     for (int i = 0; i < n; i++)  
5         for (int k = 0; k <= i; k++)  
6             if (s.substr(0, k) == s.substr(i-k+1, k))  
7                 pi[i] = k;  
8     return pi;  
9 }
```

¿Complejidad?

FUNCIÓN PREFIJO - TRIVIAL ALGORITHM

```
1  vector<int> prefix_function(string s) {  
2      int n = (int)s.length();  
3      vector<int> pi(n);  
4      for (int i = 0; i < n; i++)  
5          for (int k = 0; k <= i; k++)  
6              if (s.substr(0, k) == s.substr(i-k+1, k))  
7                  pi[i] = k;  
8      return pi;  
9  }
```

¿Complejidad? $O(n^3)$

FUNCIÓN PREFIJO - TRIVIAL ALGORITHM

```
1  vector<int> prefix_function(string s) {  
2      int n = (int)s.length();  
3      vector<int> pi(n);  
4      for (int i = 0; i < n; i++)  
5          for (int k = 0; k <= i; k++)  
6              if (s.substr(0, k) == s.substr(i-k+1, k))  
7                  pi[i] = k;  
8      return pi;  
9  }
```

¿Complejidad? $O(n^3)$

¿Podrá hacerse mejor?

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 1

La función prefijo solo puede aumentar a lo más por 1.

s	s ₀	s ₁	s ₂	s ₃	...	s _{i-2}	s _{i-1}	s _i
	$\pi(i) = 2$							

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 1

La función prefijo solo puede aumentar a lo más por 1.

s	s_0	s_1	s_2	s_3	$\dots$	s_{i-2}	s_{i-1}	s_i	s_{i+1}
								$\pi(i) = 2$	$\pi(i+1) = 4$

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 1

La función prefijo solo puede aumentar a lo más por 1.

s	s_0	s_1	s_2	s_3	$\dots$	s_{i-2}	s_{i-1}	s_i	s_{i+1}
								$\pi(i) = 2$	$\pi(i+1) = 4$

¡Contradicción!

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 1

La función prefijo solo puede aumentar a lo más por 1.

s	s_0	s_1	s_2	s_3	$\dots$	s_{i-2}	s_{i-1}	s_i	s_{i+1}
								$\pi(i) = 2$	$\pi(i+1) = 4$

¡Contradicción!

Reduce complejidad a $O(n^2)$:)

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

- ▶ Si $s[i + 1] = s[\pi(i)]$, entonces $\pi(i + 1) = \pi(i) + 1$.

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

- Si $s[i + 1] = s[\pi(i)]$, entonces $\pi(i + 1) = \pi(i) + 1$.

s	s_0	s_1	s_2	s_3	$\dots$	s_{i-2}	s_{i-1}	s_i
	$\pi(i) = 3$							

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

- Si $s[i + 1] = s[\pi(i)]$, entonces $\pi(i + 1) = \pi(i) + 1$.

s	s_0	s_1	s_2	s_3	$\dots$	s_{i-2}	s_{i-1}	s_i	s_{i+1}
	$\pi(i) = 3$								

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

- Si $s[i + 1] = s[\pi(i)]$, entonces $\pi(i + 1) = \pi(i) + 1$.

s	s_0	s_1	s_2	s_3	$\dots$	s_{i-2}	s_{i-1}	s_i	s_{i+1}
								$\pi(i) = 3$	$\pi(i + 1) = 4$

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

- ▶ Si $s[i + 1] = s[\pi(i)]$, entonces $\pi(i + 1) = \pi(i) + 1$.
- ▶ Si $s[i + 1] \neq s[\pi(i)]$, entonces necesitamos el más grande $j < \pi[i]$, tal que:
 - $s[0 \dots j - 1] = s[i - j + 1 \dots i]$
 - $s[j] = s[i + 1]$

FUNCIÓN PREFIJO - KMP

Algoritmo Eficiente

Knuth y Pratt, y independientemente Morris (Knuth-Morris-Pratt) en 1977 proponen un algoritmo con dos optimizaciones a la idea trivial.

Observación 2

¡Las comparaciones de strings no son necesarias!

- ▶ Si $s[i+1] = s[\pi(i)]$, entonces $\pi(i+1) = \pi(i) + 1$.
- ▶ Si $s[i+1] \neq s[\pi(i)]$, entonces necesitamos el más grande $j < \pi[i]$, tal que:
 - $s[0 \dots j-1] = s[i-j+1 \dots i]$
 - $s[j] = s[i+1]$

Reduce la complejidad en $O(n)$:))

FUNCIÓN PREFIJO - KMP - CÓDIGO

```
1  vector< int > prefix_function(string s) {  
2      int n = s.length();  
3      vector< int > pi(n);  
4      for (int i = 1; i < n; i++) {  
5          int j = pi[i - 1];  
6          while (j > 0 && s[i] != s[j])  
7              j = pi[j - 1];  
8          if (s[i] == s[j])  
9              j++;  
10         pi[i] = j;  
11     }  
12     return pi;  
13 }
```

FUNCIÓN PREFIJO - KMP - CÓDIGO

```
1  vector< int > prefix_function(string s) {  
2      int n = s.length();  
3      vector< int > pi(n);  
4      for (int i = 1; i < n; i++) {  
5          int j = pi[i - 1];  
6          while (j > 0 && s[i] != s[j])  
7              j = pi[j - 1];  
8          if (s[i] == s[j])  
9              j++;  
10         pi[i] = j;  
11     }  
12     return pi;  
13 }
```

Pueden también encontrar una versión template en nuestro apunte [Handbook-USM](#)

Part II

RESOLVAMOS UN PROBLEMA :)

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- ▶ Si $\pi[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- ▶ Si $\pi[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - (n + 1) - n + 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- ▶ Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- ▶ Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- ▶ Si $\pi[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - (n + 1) - n + 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

$s + \$ + t = \text{hola\$hola_estas..._hola?_chao}$

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- Si $\pi[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - (n + 1) - n + 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

$s + \$ + t = \text{hola\$hola_estas..._hola?_chao}$

$s + \$ + t$	h	o	l	a	\$	h	o	l	a	_	e	s	t	a	s	.	.	.	_	h	o	l	a	?	_	c	h	a	o
π	0	0	0	0	0	1	2	3	4	0	0	0	0	0	0	0	0	0	0	1	2	3	4	0	0	0	1	0	0

BUSCAR POR UN SUBSTRING DENTRO UN STRING

Problema

Dado un string s de largo n y un string t de largo m , con $n < m$. Determinar si s aparece en t .

- Generemos el string $s + \$ + t$, $s[i] \neq \$, \forall i < n$ y $t[i] \neq \$, \forall i < m$
- Las primeras $n + 1$ posiciones corresponden al string s y $\$$.
- Si $\pi[i] = n$ con $n < i < m + n + 1$, aparece s ! :)

Extra: Para encontrar la posición en t basta con hacer $i - (n + 1) - n + 1$.

Ejemplo

$s = \text{hola}$

$t = \text{hola_estas..._hola?_chao}$

$s + \$ + t = \text{hola\$hola_estas..._hola?_chao}$

$s + \$ + t$	h	o	l	a	\$	h	o	l	a	e	s	t	a	s	.	.	.	.	h	o	l	a	?	.	c	h	a	o
π	0	0	0	0	0	1	2	3	4	0	0	0	0	0	0	0	0	0	1	2	3	4	0	0	0	1	0	0

REFERENCES I